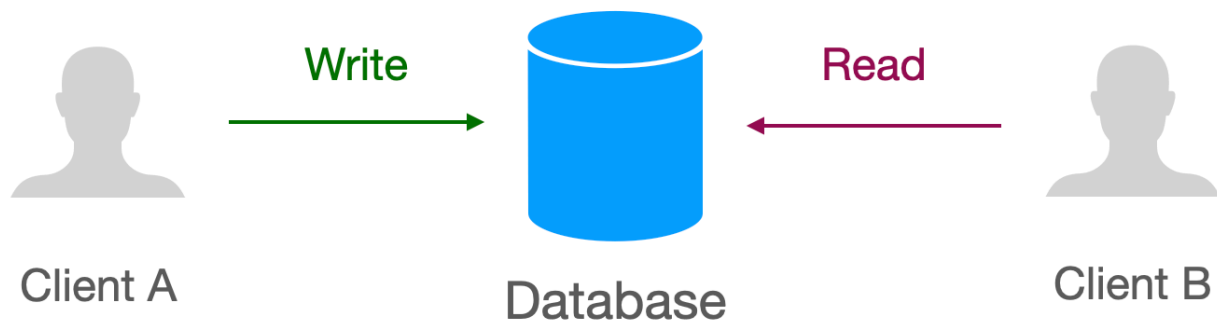


3 Efficient Ways to Enable Inter-Process Communication

 systemdesignschool.io/fundamentals/dataflow-system-design



Dataflow Overview

A user service runs on server A. An order service runs on server B. The order service needs user data to process a purchase. These processes cannot share memory across different servers. How does the order service get user information from the user service?

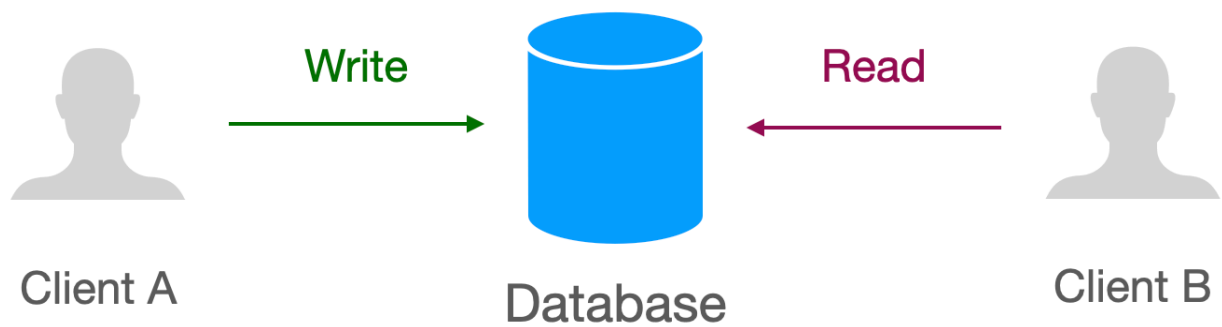
The Inter-Process Communication Problem

Processes on the same machine share memory. Read a variable. Write to an array. Direct memory access is simple and fast. But distributed systems run processes on different machines. No shared memory exists. Data must move over the network.

The order service needs to encode user data, send it over TCP, and decode it on the other end. This data movement between separate processes is dataflow. Three patterns handle this: shared databases, synchronous service calls, and message queues.

Database as Communication Channel

One process writes to a database. Another process reads from it. The database acts as an intermediary. Both processes connect to the same PostgreSQL instance. The user service inserts a row into the users table. The order service queries that table when processing an order.



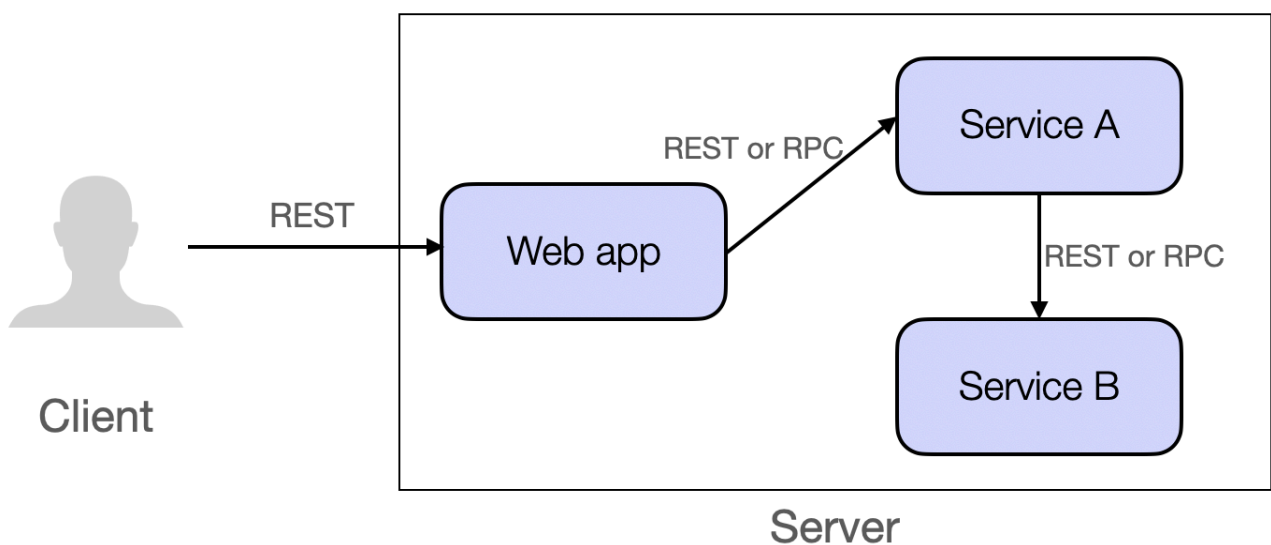
This pattern centralizes data. Multiple services read the same source of truth. Schema changes affect all readers, but at least everyone sees consistent structure. The database handles concurrency and locking. Two services updating the same row use transactions to prevent conflicts.

The database becomes a bottleneck under heavy load. Every service queries the same instance. Read replicas help but add complexity. Latency increases with each database round trip. A service chain that queries the database five times adds five network hops.

Use database dataflow when data consistency matters more than latency. Configuration data, user profiles, and inventory counts work well. Real-time chat messages do not.

Synchronous Service Calls

The order service calls the user service directly via HTTP or gRPC. Send a GET request. Wait for a response. Block until data arrives. This is request-response communication.



The client controls when to fetch data. Services expose APIs. Other services call those APIs on demand. The order service requests user data only when needed for an active order. No polling. No background updates. Just direct requests.

Each service encapsulates its data. The user service owns user data. The order service cannot query the user database directly. It must go through the user service API. This isolation allows independent evolution. The user service can change its database schema without affecting callers as long as the API contract remains stable.

The caller blocks waiting for a response. If the user service takes 200ms to respond, the order service waits 200ms. Chain multiple calls and latency compounds. Service A calls B. B calls C. C calls D. Total latency is the sum of all calls plus network overhead.

One service down breaks the chain. If the user service crashes, the order service cannot fetch user data. The order fails even though the order service itself works fine. [Timeouts](#), [retries](#), and [circuit breakers](#) mitigate these failures but add complexity.

Use synchronous calls when you need immediate data to proceed. Payment processing needs confirmation now. User authentication blocks until credentials are verified. Real-time queries require current data.

Message Queues for Asynchronous Flow

The order service publishes an event: "OrderCreated." It does not wait for a response. The message goes to a broker like Kafka or RabbitMQ. The broker stores the message. The inventory service subscribes to OrderCreated events. When ready, it processes the message and reserves stock.



The sender and receiver decouple. The order service does not know who consumes its events. Multiple services can subscribe. The inventory service reserves stock. The notification service sends an email. The analytics service logs the event. One message serves many consumers.

The broker buffers messages. If the inventory service is slow, messages queue up. The order service continues publishing. Temporary slowdowns do not block senders. The broker provides fault tolerance. If a consumer crashes, messages wait until it recovers.

Messages flow one way. The sender publishes and forgets. It does not get a response. For two-way communication, the receiver must publish a separate response message. This eventual consistency works when immediate confirmation is not required.

Use message queues when services operate at different speeds or when one event triggers multiple actions. Background jobs, notifications, and data pipeline events fit this pattern. Do not use message queues when you need an immediate response or strong consistency.

Comparing Patterns

Pattern	Recipients	Sync/Async	Response	Use When
Database	Many	Async	None	Centralized data, multiple readers
Service Calls	One	Sync	Immediate	Need data now to proceed
Message Queue	Many	Async	None	Decouple producers and consumers

Database dataflow suits read-heavy workloads with consistent schemas. Service calls work for interactive requests needing real-time responses. Message queues excel at asynchronous processing with variable consumer speeds.

Real systems combine patterns. An e-commerce platform uses service calls for checkout, message queues for order processing, and databases for product catalogs. Choose based on latency requirements, consistency needs, and coupling tolerance.

From here, push versus pull models in dataflow lead into how Twitter implements both patterns at scale.